Comparison between Dijkstra and A* algorithms for motion planning

Vadacca Lorenzo (331703)

I. Introduction

Motion planning is a fundamental task in autonomous vehicle navigation. It consists in computing a valid and efficient path from a given start configuration to a desired goal, considering constraints like road structure and vehicle dynamics. In global motion planning, this problem is tackled using algorithms such as Dijkstra and A*.

The purpose of this report is to analyze and compare the two algorithms by applying them to the road network of a real city, using data obtained from OpenStreetMap and processed via the *osmnx* library in *Python*. The comparison focuses on theoretical aspects, code structure, and empirical performance, with a particular focus on the number of iterations required to reach the goal. Additionally, graphical visualizations have been implemented, including side-by-side heatmaps to highlight the spatial differences in how each algorithm explores the graph.

II. Theoretical background

A. Dijkstra's algorithm

Dijkstra's algorithm finds the minimum-weight path from a start node to all other nodes in a weighted directed graph. It works by maintaining a priority queue of unvisited nodes, always expanding the node with the lowest cumulative cost $g(n)$ at each step of the search.
Considering its limitations, in case of large graphs, a large number of nodes may be visited, resulting in high computational cost that may not be acceptable for real-time applications (e.g. using Google Maps during a car trip).

B. A* algorithm

A* algorithm extends Dijkstra's by introducing a heuristic function $h(n)$ that estimates the cost from any node n to the goal. The total score for a node then becomes:

$$f(n) = g(n) + h(n) \tag{1}$$

where:

- $g(n)$ is the cost from the start to node n
- $h(n)$ is a heuristic estimate from n to the goal

This prioritization leads A* to explore fewer nodes, especially when using an admissible heuristic (one that never overestimates the true cost to reach the goal).

III. Code structure and functionality

A. Importing libraries (lines 1-5)

`osmnx`, `random`, `heapq`, `math`, `matplotlib.pyplot` libraries are imported to manage key functionalities in the motion planning pipeline: `osmnx` is used to download, project, and visualize street networks from OpenStreetMap; `random` selects variable origin and destination nodes; `heapq` provides efficient priority queue operations for Dijkstra and A*; `math` supports heuristic computation through functions like square root; and `matplotlib.pyplot` is used to generate the final side-by-side heatmaps and scatter plots of the heuristic values across iterations.

B. Styling edges (lines 7–26)

These functions assign graphical properties (color, thickness, transparency) to edges, used for real-time visualization of the algorithm's behavior.

C. Visualizing graph (lines 27–36)

The function uses `osmnx` to plot the graph with the styling attributes dynamically assigned during algorithm execution.

D. Heuristic function (lines 38–44)

The heuristic used in A* estimates the travel time to the goal based on the Euclidean distance between projected coordinates:

$$h(n) = \frac{\text{distance}(n, \text{goal})}{\text{avg_speed}}$$

with average speed set to 50 km/h (13.89 m/s).

E. Dijkstra's Algorithm (lines 46–76)

Firstly the function resets all node attributes, sets origin distance to 0 and marks both origin and destination with increased node size for visualization.
Afterwards, it uses a min-priority queue (implemented via a heap) to select the node with the lowest cumulative cost, skipping the nodes which have already been visited, if any. Going on, for each neighbor, if a shorter path is found, it updates the distance and the path, highlighting active and visited edges for real-time feedback. The function terminates when the goal is reached, printing in output the number of iterations (steps) necessary to accomplish the task.

```
def dijkstra(orig, dest, plot=False):
    for node in G.nodes:
        G.nodes[node]["visited"] = False
        G.nodes[node]["distance"] = float("inf")
        G.nodes[node]["previous"] = None
        G.nodes[node]["size"] = 0
    for edge in G.edges:
        style_unvisited_edge(edge)
    G.nodes[orig]["distance"] = 0
    G.nodes[orig]["size"] = 50
    G.nodes[dest]["size"] = 50
    pq = [(0, orig)]
    step = 0
    while pq:
        _, node = heapq.heappop(pq)
        if node == dest:
            print("Iterations:", step)
            return
        if G.nodes[node]["visited"]: continue
        G.nodes[node]["visited"] = True
        for edge in G.out_edges(node):
            style_visited_edge((edge[0], edge[1],
            ↪  0))
            neighbor = edge[1]
            weight = G.edges[(edge[0], edge[1],
            ↪  0)]["weight"]
            if G.nodes[neighbor]["distance"] >
            ↪  G.nodes[node]["distance"] + weight:
                G.nodes[neighbor]["distance"] =
                ↪  G.nodes[node]["distance"] +
                ↪  weight
                G.nodes[neighbor]["previous"] =
                ↪  node
                heapq.heappush(pq,
                ↪  (G.nodes[neighbor]["distance"],
                ↪  neighbor))
                for edge2 in G.out_edges(neighbor):
                    style_active_edge((edge2[0],
                    ↪  edge2[1], 0))
        step += 1
```

F. A* algorithm (lines 78–113)

The initialization phase is quite similar to Dijkstra, but in this case the function includes f_score initialization and the origin variable is inserted into open_set with its f_score. Afterwards, the function pops the node with the lowest f_score and, for each neighbor, it computes a tentative g_score and it compares it to the current best-known distance, updating previous, distance, and f_score attributes. If the new path is shorter, it updates the node and adds it to the open set. and, like Dijkstra, it uses edge styling to visualize steps.

```
def a_star(orig, dest, maxs, plot=False):
    for node in G.nodes:
        G.nodes[node]["visited"] = False
        G.nodes[node]["distance"] = float("inf")
        G.nodes[node]["f_score"] = float("inf")
        G.nodes[node]["previous"] = None
        G.nodes[node]["size"] = 0
    for edge in G.edges:
        style_unvisited_edge(edge)
    G.nodes[orig]["distance"] = 0
    G.nodes[orig]["f_score"] = heuristic(orig,
    ↪  dest)
    G.nodes[orig]["size"] = 50
    G.nodes[dest]["size"] = 50

    open_set = [(G.nodes[orig]["f_score"], orig)]
    step = 0
    while open_set:
        _, node = heapq.heappop(open_set)
        if node == dest:
            print("Iterations:", step)
            return
        if G.nodes[node]["visited"]: continue
        G.nodes[node]["visited"] = True
        for edge in G.out_edges(node):
            style_visited_edge((edge[0], edge[1],
            ↪  0))
            neighbor = edge[1]
            weight = G.edges[(edge[0], edge[1],
            ↪  0)]["weight"]
            tentative_g_score =
            ↪  G.nodes[node]["distance"] + weight
            if tentative_g_score <
            ↪  G.nodes[neighbor]["distance"]:
                G.nodes[neighbor]["previous"] =
                ↪  node
                G.nodes[neighbor]["distance"] =
                ↪  tentative_g_score
                G.nodes[neighbor]["f_score"] =
                ↪  tentative_g_score +
                ↪  heuristic(neighbor, dest)
                heapq.heappush(open_set,
                ↪  (G.nodes[neighbor]["f_score"],
                ↪  neighbor))
                for edge2 in G.out_edges(neighbor):
                    style_active_edge((edge2[0],
                    ↪  edge2[1], 0))
        step += 1
```

G. Reconstructing path (lines 116–130)

This function follows the previous pointers from destination back to origin, then calculates the total distance, highlighting the developed path in white, and finally it increments the usage counter for heatmap visualization.

H. Plotting heatmap (lines 131–145)

This function uses `osmnx.plot_graph()` in combination with `get_edge_colors_by_attr()` to visualize the number of times each edge was used by the selected algorithm (`dijkstra` or `a_star`). Edges are colored using a colormap based on the attribute `algorithm_uses`.

Additionally, only the nodes marked as `visited` during the algorithm's execution are shown in green, while all others are hidden by setting their size to zero and color to "none". The background color is set to dark gray (`#18080e`) for visual consistency. The function uses `matplotlib` in order to allow side-by-side comparisons of the Dijkstra and A* heatmaps within a single figure.

I. Constructing the graph and executing the main function (lines 147–193)

The graph is created by downloading the street network of the city of Turin using OSMnx, specifying `network_type="drive"` to include only drivable roads. Immediately after, the graph is projected into a metric coordinate reference system using `ox.project_graph()`, so that all geometric computations (e.g., distances) are performed in meters and not in degrees (dealing with latitude and longitude).

For each edge, the attribute `maxspeed` is cleaned: if it is a list or string, the script extracts the numerical value and

converts it to meters per second. If the value is missing or invalid, a default of 50 km/h (13.89 m/s) is used. The attribute `weight` is then computed as the ratio between `length` (in meters) and `maxspeed` (in m/s), representing an estimated travel time in seconds, and stored for use in path computation.

Each edge is initialized with usage counters for both Dijkstra and A* (`dijkstra_uses`, `a_star_uses`), set to zero. Two random nodes from the graph are then selected as the origin and destination.

Finally, both algorithms are executed in sequence. Their respective paths are reconstructed and highlighted using edge styling, and heatmaps of the explored areas are plotted side by side using custom subplots. The background color is set to dark for visual consistency, and visited nodes are shown in green, so that the entity of the computational cost between the two algorithms can be perceived in advance.

IV. Validation and conclusions

To validate the performance of the implemented algorithms, two random nodes are selected as origin and destination within the street network of Turin, Italy. The number of iterations required to reach the goal is printed for both Dijkstra and A*. The computed paths are visualized using heatmaps, and visited nodes are highlighted in green.

A. Example

The Turin road network is downloaded from OpenStreetMap using the OSMnx library. The graph is then projected to a metric coordinate system to allow realistic geometric computations. After assigning edge weights based on estimated traversal time (i.e., edge length divided by speed), the algorithms were executed for the following randomly selected nodes:

```
Origin ID: 255281793
Destination ID: 728492344
Running Dijkstra
Iterations: 7781
Running A*
Iterations: 2012
```



Fig. 1. Side-by-side heatmaps: Dijkstra (left) and A* (right). Edge usage intensity is color-coded using a hot colormap; visited nodes are shown in green.

B. Observations

As shown, A* required significantly fewer iterations than Dijkstra. Despite this, both algorithms produced the same shortest path, confirming that the heuristic used in A* is admissible and consistent. This allowed the algorithm to converge more efficiently by focusing the search toward the destination.

Additionally, the visualizations clearly demonstrate that A* explores fewer irrelevant areas of the graph. The presence of visited nodes (green), along with the density of active (red) and final-path (white) edges, confirms the improved spatial focus achieved by A*

C. Conclusion

Both Dijkstra and A* guarantee optimal path computation. However, A* exhibits superior performance in terms of iteration efficiency and spatial selectivity. Thanks to its heuristic component based on euclidean travel time, it effectively reduces the search space. These characteristics make A* particularly suitable for real-time applications such as autonomous driving.